Solution du TD N° 3 (détaillée)

Systèmes de numération et codage de l'information

Ex1 :

Convertir en décimale :

$$(100011011011)_2 = 1\times2^{11} + 0\times2^{10} + 0\times2^{9} + 0\times2^{8} + 1\times2^{7} + 1\times2^{6} + 0\times2^{5} + 1\times2^{4} + 1\times2^{3} + 0\times2^{2} + 1\times2^{1} + 1\times2^{0}$$

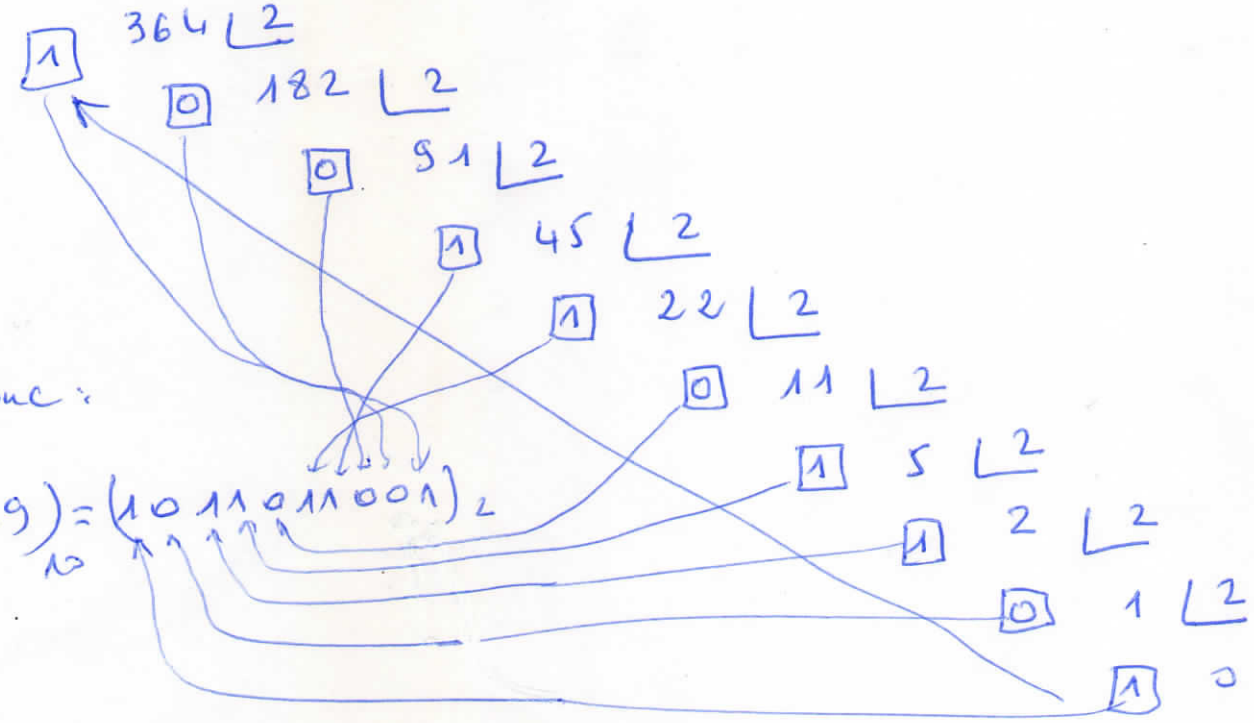
$$= 2048 + 0 + 0 + 0 + 128 + 64 + 0 + 16 + 8 + 0 + 2 + 1$$

$$= (2267)_{10}$$

Ex2 :

Convertir $(729)_{10}$ en binaire en divisant par 2 (binaire)

729 | 2



donc :

$$(729)_{10} = (1011011001)_2$$

Ex3 :

conversion de $(5431)_8$ en binaire :

on représente chaque chiffre séparément en binaire (sur 3 bits).

on a

chiffre octal	0	1	2	3	4	5	6	7
Equivalent binaire	000	001	010	011	100	101	110	111

donc



$$\Rightarrow (5431)_8 = (101100011001)_2$$

Ex4 :

convertir $(614)_8$ en décimal.

$(614)_8 = 6 \times 8^2 + 1 \times 8^1 + 4 \times 8^0 = 384 + 8 + 4 = (396)_{10}$

Ex5 :

Dans la base 8 on a :

$(626)_8 + 1 = (627)_8$, $(627)_8 + 1 = (630)_8$, $(630)_8 + 1 = (631)_8$

donc les 3 nombres à la suite de $(626)_2$ sont : $(627)_8$, $(630)_8$ et $(631)_8$

Ex6 :

- conversion de $(146)_{10}$ en octal (en divisant par 8)

146 | 8

66 18 | 8

2 2 | 8

2 0

$(146)_{10} = (222)_8$

- conversion de $(222)_8$ en binaire :

on a 2 2 2 donc $(222)_8 = (10010010)_2$

↓ ↓ ↓

010 010 010

Ex7 :

- conversion de $(10011101)_2$ en octal.

10011101 donc $(10011101)_2 = (235)_8$

↓ ↓ ↓

2 3 5

Ex8 :

- conversion de $(24CE)_{16}$ en décimal

$(24CE)_{16} = 2 \times 16^3 + 4 \times 16^2 + C \times 16^1 + E \times 16^0$

$= 2 \times 4096 + 4 \times 256 + 12 \times 16 + 14 \times 1$

$= 8192 + 1024 + 192 + 14$

$(24CE)_{16} = (9422)_{10}$

Ex9 :

- Conversion de $(3117)_{10}$ en Hexadécimal (en divisant par 16)

3117 | 16

151 194 | 16

77 34 12 | 16

D ← 13 2 12 0

C

donc $(3117)_{10} = (C2D)_{16}$

- Conversion de $(C2D)_{16}$ en binaire

en hexadecimal on représente chaque chiffre en son equivalent binaire de 4 bits.

donc C 2 D

1100 0010 1101

$(C2D)_{16} = (110000101101)_2$

Ex10 :

- conversion de $(1001000000110101)_2$ en hexadécimal.

pour cela on divise le nombre binaire en groupes de 4 bits.

puis on remplace chaque groupe en son equivalent hexadecimal.

donc

1001 0000 0011 0101

9 0 3 5

$(1001000000110101)_2 = (9035)_{16}$

Ex11 :

Dans la base 16 on a

$(E9D)_{16} + 1 = (E9E)_{16}$, $(E9E)_{16} + 1 = (E9F)_{16}$,

$(E9F)_{16} + 1 = (EA0)_{16}$, $(EA0)_{16} + 1 = (EA1)_{16}$

donc les 4 nombres = la suite de $(E9D)_{16}$ sont :

$(E9E)_{16}$, $(E9F)_{16}$, $(EA0)_{16}$ et $(EA1)_{16}$

Ex12 :

- conversion de $(312,4)_5$ en décimal.

$$(312,4)_5 = 3\times5^2 + 1\times5^1 + 2\times5^0 + 4\times5^{-1}$$
$$= 3\times25 + 5 + 2 + \frac{4}{5}$$
$$= 75 + 5 + 2 + 0,8 = (82,8)_{10}$$

Ex13 :

- Conversion de $(673,6)_8$ en binaire puis en hexadécimal :

6 7 3 , 6
110 111 011 , 110

donc $(673,6)_8 = (110111011,110)_2$

on ajoute des zéros ↓ 000110111011,1100 ← on ajoute 2 zéros pour avoir 4 bits

1 B B , C

donc $(673,6)_8 = (110111011,110)_2 = (1BB,C)_{16}$

- Conversion de $(E7C,B)_{16}$ en binaire puis en octal :

E 7 C , B
1110 0111 1100 , 1011

donc $(E7C,B)_{16} = (111001111100,1011)_2$

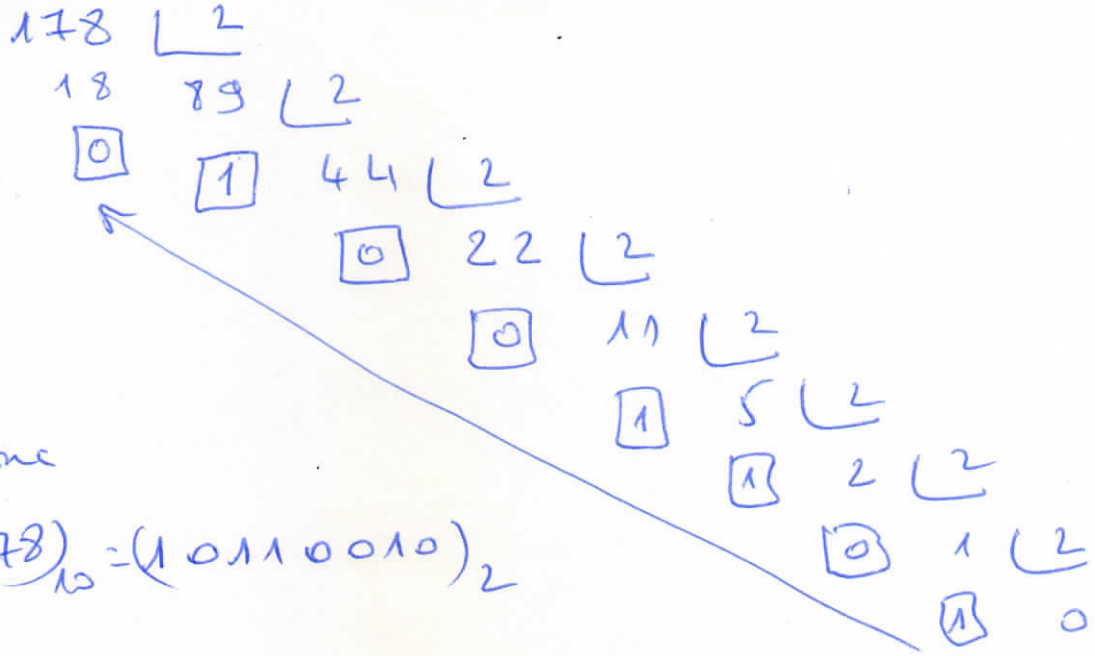
111001111100,1011 ← on peut l'ignorer ou on peut ajouter 2 zéros pour avoir 4

7 1 7 6 , 5

donc $(E7C,B)_{16} = (111001111100,1011)_2 = (7176,5)_8$

Ex14 :

le code binaire pur de $(178)_{10}$ en divisant par 2



donc

$(178)_{10} = (10110010)_2$

- le code BCD de $(178)_{10}$: on change chaque chiffre par son equivalent binaire.

1 7 8

0001 0111 1000

donc $(178)_{10} = (000101111000)_{BCD}$

EX15 :

on ajoute, le zéro signifie positif

* $(13)_{10} = (1101)_2 \Rightarrow +13 = (01101)_2$ sur 5 bits
* $(9)_{10} = (1001)_2 \Rightarrow +9 = (01001)_2$ sur 5 bits

(-9) on realise le complement à 2 de $(+9)$, d'abord on réalise le complément à 1 en changeant chaque 0 par 1 et chaque 1 par 0. donc $01001 \xrightarrow{\text{son complement à 1}} 10110$

puis le complement à 2 en ajoutant un 1.

$10110 + 1 = 10111$ donc $-9 = (10111)_2$

↑ signe négatif

* $(3) = (00011)_2 \Rightarrow +3 = 00011$
* (-3) on applique la même méthode que (-9). on aura complement à 1 de 3 est 11100, le complement à 2 est

$11100 + 1 = 11101$ donc

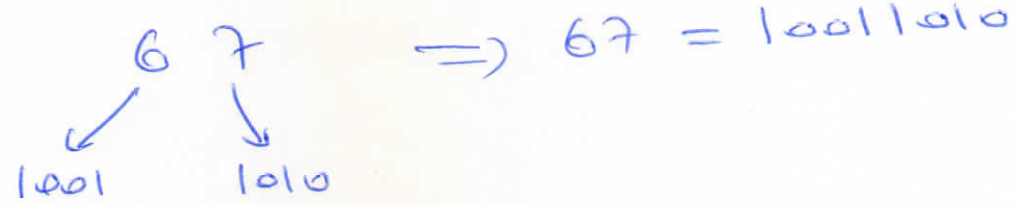
$-3 = (11101)_2$

Ex 16 :

code Excédant 3 (excess 3) : on obtient en ajoutant 3 au code binaire exemple : 2 = 0010 + 0011 = 0101
                                                                                                  03

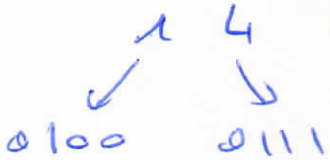
Décimal	0	1	2	3	4	5	6	7	8	9
Excess 3	0011	0100	0101	0110	0111	1000	1001	1010	1011	1100

* 67 en code excess 3 :



$\Rightarrow 67 = 10011010$

* 14 en code excess 3 :



$\Rightarrow 14 = 01000111$

Ex 17 :

Si on additionne les codes BCD dont la somme ne dépasse pas 9, on aura un résultat correcte de la somme BCD (pas de correction)

exemple : $4 \rightarrow 0100$, $+3 \rightarrow +0011$

$0111 \Rightarrow 7 < 9$

Mais si la somme dépasse 9, il faut alors apporter une correction c'est à dire on ajoute 6 (0110) au résultat

exemple : $8 \rightarrow 1000$, $+5 \rightarrow +0101$

$1101 \Rightarrow 13 > 9$

donc on ajoute 6 : $+\ 0110$

$0001\ 0011$

1 3

* 59 → 0 1 1 0 1 1 0 0 1

\+ 38 → + 0 0 1 1 1 0 0 0

1 0 0 1 0 0 0 1 ⇒ supérieur à 9

donc on ajoute 6 + 0 1 1 0

1 0 0 1 0 1 1 1

donc 59 + 38 = 1 0 0 1 0 1 1 1

* 275 → 0 0 1 0 0 1 1 1 0 1 0 1

\+ 641 → 0 1 1 0 0 1 0 0 0 0 0 1

1 0 0 0 1 0 1 1 0 1 1 0 → < 9

< 9 > 9

on ajoute 6. 1 0 0 0 1 0 1 1 0 1 1 0

donc + 0 1 1 0

1 0 0 1 0 0 0 1 0 1 1 0

donc

275 + 641 = 1 0 0 1 0 0 0 1 0 1 1 0

Ex18 :

Dans cette exercice on utilise le tableau du code ASCII (voir cours).

décodage de la séquence de bits suivantes :

1 0 1 0 0 1 1 1 0 1 0 1 0 0 1 0 0 1 1 1 1 0 1 0 0 0 0

5 3 5 4 4 F 5 0

S T O P

Ex19 :

En utilisant le tableau du code ASCII. (voir cours) :

la forme hexadécimale du message : COUT=72 $

(C) = 43 ; (O) = 4F ; (U) = 55 ; (T) = 54 ; (=) = 3D ; (7) = 37

(2) = 32 ; (␣) = SP = 20 ; ($) = 24

Ex20 :

Rappel :

Addition Logique	0+0=0	0+1=1	1+0=1	1+1=1
Addition Arithmétique	0+0=0	0+1=1	1+0=1	1+1=10

retenu

```
 1 0 1 0 1        0 0 0 1 1 1
+ 0 0 1 1       + 1 1 0 1 1 0
-------------    -------------
 1 0 0 0 0       1 1 1 1 0 1
```

Ex21 :

```
  1101
- 1010   ---complément à 1--> 0101   ---complément à 2--> 0101 + 1 = 0110
  ????
```

donc

```
  1101
+ 0110
```

résultat st : 0011

on néglige le bit ↗ (0011)

```
  11011
- 11001   ---c. à 1--> 00110   ---c. à 2--> 00110 + 1 = 00111
  ?????
```

résultat st : 00010

donc

```
  11011
+ 00111
-------
  00010
```

```
  01000
- 0010   --> 00010   ---c à 1--> 11101   ---c à 2--> 11110
  ?????
```

résultat st : 00110

donc

```
  01000
+ 11110
-------
  00110
```

```
  10111
- 11111   ---c à 1--> 00000   ---c. à. 2--> 00001
  ?????
```

résultat st : 11000

donc

```
  10111
+ 00001
-------
  11000
```